DATABASE AUTOMATION USING VIEWS AND PROCEDURES

AIM:

To implement database automation in the ApexMart E-Commerce Database System using Views, Stored Procedures, and Functions to simplify database operations and improve efficiency.

SCENARIO:

ApexMart is an e-commerce company that frequently generates reports and performs repetitive database operations such as order processing, sales calculation, inventory management, and customer analysis.

To improve efficiency and reduce repeated SQL coding, the company uses database automation techniques such as Views, Stored Procedures, and Functions.

These database objects help automate order processing, calculate sales, manage product stock, and generate customer and sales reports.

1.Create Views for Frequently Used Reports:

A View is a virtual table created from one or more SQL queries. It stores the query definition and provides simplified access to frequently used information.

The following views are created in the ApexMart database:

1. Customer Order Report View:

Displays customer name, order ID, order date, total amount, and order status. It helps retrieve customer order information easily.

```
SQL> CREATE OR REPLACE VIEW VW_SHOPPER_ORDER_SUMMARY AS
  2  SELECT s.shopper_name, o.order_num, o.booking_date, o.order_total, o.fulfilment_state
  3  FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id;

View created.
```

2. Product Sales Report View:

Displays product name, quantity sold, and total revenue. It helps identify product sales performance.

```
SQL> CREATE OR REPLACE VIEW VW_ITEM_PERFORMANCE_METRICS AS
  2  SELECT p.item_name, NVL(SUM(od.item_qty), 0) AS quantity_sold, NVL(SUM(od.item_qty * od.unit_rate), 0) AS total_revenue
  3  FROM catalog_item p LEFT JOIN order_line_item od ON p.item_code = od.item_code
  4  GROUP BY p.item_code, p.item_name;

View created.
```

3. Payment Status View:

Displays order ID, payment mode, payment status, and transaction amount. It helps monitor payment information.

```
SQL> CREATE OR REPLACE VIEW VW_BILLING_TRANSACTION_AUDIT AS
  2  SELECT order_num, pay_channel, clearance_status, settled_amount
  3  FROM order_billing;

View created.
```

2.Develop Stored Procedures for Order Processing:

A Stored Procedure is a collection of SQL statements stored in the database that can be executed whenever required. It helps automate repeated database operations.

The following procedures are developed:

1. Add New Customer Order:

- Insert new order information.
- Add purchased product details.
- Calculate the total order amount.

```
SQL> CREATE OR REPLACE PROCEDURE SP_REGISTER_SALES_TRANSACTION(
  2    p_order_num IN VARCHAR2, p_shopper_id IN VARCHAR2, p_item_code IN VARCHAR2,
  3    p_qty IN NUMBER, p_txn_ref IN VARCHAR2, p_channel IN VARCHAR2
  4  ) AS
  5    v_rate NUMBER(10,2); v_total NUMBER(10,2);
  6  BEGIN
  7    SELECT unit_rate INTO v_rate FROM catalog_item WHERE item_code = p_item_code;
  8    v_total := v_rate * p_qty;
  9    INSERT INTO sales_order(order_num, shopper_id, booking_date, order_total, fulfilment_state)
 10    VALUES(p_order_num, p_shopper_id, SYSDATE, v_total, 'Pending');
 11    INSERT INTO order_line_item(line_id, order_num, item_code, item_qty, unit_rate)
 12    VALUES('LN-'||SUBSTR(p_order_num, 4), p_order_num, p_item_code, p_qty, v_rate);
 13    INSERT INTO order_billing(txn_ref, order_num, pay_channel, clearance_status, settled_amount)
 14    VALUES(p_txn_ref, p_order_num, p_channel, 'Pending', v_total);
 15    COMMIT;
 16  END;
 17  /

Procedure created.
```

2. Update Order Status:

Updates the status of an order based on the order ID. The order status can be changed from Pending to Shipped and Delivered.

```
SQL> CREATE OR REPLACE PROCEDURE SP_UPDATE_FULFILMENT_STATE(p_order_num IN VARCHAR2, p_new_state IN VARCHAR2) AS
  2  BEGIN
  3    UPDATE sales_order SET fulfilment_state = p_new_state WHERE order_num = p_order_num;
  4    IF SQL%ROWCOUNT = 0 THEN
  5      RAISE_APPLICATION_ERROR(-20002, 'Order Number Not Found');
  6    END IF;
  7    COMMIT;
  8  END;
  9  /

Procedure created.
```

3. Generate Customer Purchase History:

Accepts Customer ID as input and displays the customer's order details, purchase amount, order date, and order status.

```
SQL> CREATE OR REPLACE PROCEDURE SP_RETRIEVE_SHOPPER_ORDERS(p_shopper_id IN VARCHAR2, p_result OUT SYS_REFCURSOR) AS
  2  BEGIN
  3    OPEN p_result FOR
  4      SELECT s.shopper_name, o.order_num, o.booking_date, o.order_total, o.fulfilment_state
  5      FROM shopper_account s JOIN sales_order o ON s.shopper_id = o.shopper_id
  6      WHERE s.shopper_id = p_shopper_id;
  7  END;
  8  /

Procedure created.
```

3.Create Functions for Sales Calculations:

A Function performs a specific calculation and returns a value.

The following functions are created:

1. Calculate Total Sales:

Accepts a date range as input and returns the total revenue generated during that period.

```
SQL> CREATE OR REPLACE FUNCTION FN_COMPUTE_INTERVAL_REVENUE(p_start_date IN DATE, p_end_date IN DATE) RETURN NUMBER AS
  2    v_total NUMBER;
  3  BEGIN
  4    SELECT NVL(SUM(order_total), 0) INTO v_total FROM sales_order
  5    WHERE TO_DATE(booking_date, 'DD-MON-YY') >= p_start_date AND TO_DATE(booking_date, 'DD-MON-YY') <= p_end_date;
  6    RETURN v_total;
  7  END;
  8  /

Function created.
```

2. Calculate Customer Total Spending:

Accepts Customer ID as input and returns the total amount spent by that customer.

```
SQL> CREATE OR REPLACE FUNCTION FN_GET_SHOPPER_EXPENDITURE(p_shopper_id IN VARCHAR2) RETURN NUMBER AS
  2    v_spending NUMBER;
  3  BEGIN
  4    SELECT NVL(SUM(order_total), 0) INTO v_spending FROM sales_order WHERE shopper_id = p_shopper_id;
  5    RETURN v_spending;
  6  END;
  7  /

Function created.
```

3. Calculate Product Revenue:

Accepts Product ID as input and returns the total revenue generated by that product.

```
SQL> CREATE OR REPLACE FUNCTION FN_EVALUATE_ITEM_REVENUE(p_item_code IN VARCHAR2) RETURN NUMBER AS
  2    v_revenue NUMBER;
  3  BEGIN
  4    SELECT NVL(SUM(item_qty * unit_rate), 0) INTO v_revenue FROM order_line_item WHERE item_code = p_item_code;
  5    RETURN v_revenue;
  6  END;
  7  /

Function created.
```

4.Automate Repetitive Database Operations:

Database automation helps perform repeated operations automatically and reduces manual work.

1. Order Processing Automation:

When a customer places an order:

- Insert order details.
- Reduce product stock.
- Create a payment record.
- Update order status.

```
SQL> CREATE OR REPLACE TRIGGER TRG_CHECK_ITEM_INVENTORY
  2  BEFORE INSERT ON order_line_item FOR EACH ROW
  3  DECLARE v_stock NUMBER;
  4  BEGIN
  5    SELECT available_stock INTO v_stock FROM catalog_item WHERE item_code = :NEW.item_code;
  6    IF v_stock < :NEW.item_qty THEN
  7      RAISE_APPLICATION_ERROR(-20001, 'Insufficient Catalog Item Inventory');
  8    END IF;
  9  END;
 10  /

Trigger created.
```

```
SQL> CREATE OR REPLACE TRIGGER TRG_DEDUCT_ITEM_INVENTORY
  2  AFTER INSERT ON order_line_item FOR EACH ROW
  3  BEGIN
  4    UPDATE catalog_item SET available_stock = available_stock - :NEW.item_qty
  5    WHERE item_code = :NEW.item_code;
  6  END;
  7  /

Trigger created.
```

2. Inventory Update Automation:

The inventory operations include:

- Increase stock after receiving new supply.
- Reduce stock after a purchase.
- Identify products with low stock.

```
SQL> CREATE OR REPLACE PROCEDURE SP_INWARD_INVENTORY_RESTOCK(p_item_code IN VARCHAR2, p_restock_qty IN NUMBER) AS
  2  BEGIN
  3    IF p_restock_qty <= 0 THEN
  4      RAISE_APPLICATION_ERROR(-20003, 'Restock Quantity Must Be Positive');
  5    END IF;
  6    UPDATE catalog_item SET available_stock = available_stock + p_restock_qty WHERE item_code = p_item_code;
  7    IF SQL%ROWCOUNT = 0 THEN
  8      RAISE_APPLICATION_ERROR(-20004, 'Item Code Not Found');
  9    END IF;
 10    COMMIT;
 11  END;
 12  /

Procedure created.
```

```
SQL> CREATE OR REPLACE VIEW VW_DEPLETED_INVENTORY_ITEMS AS
  2  SELECT item_code, item_name, available_stock FROM catalog_item WHERE available_stock <= 5;

View created.
```

3. Report Generation Automation:

The system generates the following reports:

- Daily sales report.

```
SQL> CREATE OR REPLACE VIEW VW_DAILY_TURNOVER_SUMMARY AS
  2  SELECT TRUNC(TO_DATE(booking_date, 'DD-MON-YY')) AS sales_date,
  3  COUNT(order_num) AS total_orders, NVL(SUM(order_total), 0) AS total_revenue
  4  FROM sales_order GROUP BY TRUNC(TO_DATE(booking_date, 'DD-MON-YY'));

View created.
```

- Monthly revenue report.

```
SQL> CREATE OR REPLACE VIEW VW_MONTHLY_REVENUE_AUDIT AS
  2  SELECT TO_CHAR(TO_DATE(booking_date, 'DD-MON-YY'), 'YYYY-MM') AS sales_month,
  3  COUNT(order_num) AS total_orders, NVL(SUM(order_total), 0) AS total_revenue
  4  FROM sales_order GROUP BY TO_CHAR(TO_DATE(booking_date, 'DD-MON-YY'), 'YYYY-MM');

View created.
```

- Customer purchase report.

```
SQL> CREATE OR REPLACE VIEW VW_SHOPPER_LIFETIME_EXPENDITURE AS
  2  SELECT s.shopper_id, s.shopper_name, COUNT(o.order_num) AS total_orders, NVL(SUM(o.order_total), 0) AS total_spending
  3  FROM shopper_account s LEFT JOIN sales_order o ON s.shopper_id = o.shopper_id
  4  GROUP BY s.shopper_id, s.shopper_name;

View created.
```

5.Test Database Objects:

The created database objects are tested to verify their correctness.

Views Testing:

- Check correct data retrieval.
- Verify report accuracy.

```
SQL> SELECT * FROM VW_SHOPPER_ORDER_SUMMARY;

SHOPPER_NAME                   ORDER_NUM  BOOKING_D ORDER_TOTAL FULFILMENT_
------------------------------ ---------- --------- ----------- -----------
Aarav Sharma                   SO-901     01-SEP-26      560.00 Delivered
Meera Krishnan                 SO-902     02-SEP-26      260.00 Delivered
Rohan Desai                    SO-903     03-SEP-26      650.00 Shipped
Pooja Nair                     SO-904     04-SEP-26      420.00 Delivered
Vikram Sengupta                SO-905     05-SEP-26      190.00 Pending
Ananya Iyer                    SO-906     06-SEP-26      890.00 Shipped
Karthik Raman                  SO-907     07-SEP-26      460.00 Delivered
Divya Nambiar                  SO-908     08-SEP-26      380.00 Pending
Siddharth Verma                SO-909     09-SEP-26      240.00 Pending
Tanvi Kulkarni                 SO-910     10-SEP-26      560.00 Shipped

10 rows selected.
```

```
SQL> SELECT * FROM VW_ITEM_PERFORMANCE_METRICS;

ITEM_NAME                      QUANTITY_SOLD TOTAL_REVENUE
------------------------------ ------------- -------------
Ergonomic Desk Mat                         5       1350.00
Braided Type-C Cable                       3        360.00
Stainless Thermal Tumbler                  1        340.00
Mechanical Numpad                          1        420.00
Ceramic Aroma Diffuser                     1        260.00
Aluminum Laptop Riser                      1        650.00
Noise-Canceling Earbuds                    1        890.00
Compact Desk Fan                           1        190.00
Canvas Tech Organizer                      1        150.00
Artisanal Herbal Tea                       0          0.00

10 rows selected.
```

```
SQL> SELECT * FROM VW_BILLING_TRANSACTION_AUDIT;

ORDER_NUM  PAY_CHANNEL          CLEARANCE_STATUS SETTLED_AMOUNT
---------- -------------------- ---------------- --------------
SO-901     UPI                  Settled                 560.00
SO-902     Debit Card           Settled                 260.00
SO-903     Net Banking          Settled                 650.00
SO-904     UPI                  Settled                 420.00
SO-905     Cash On Delivery     Pending                 190.00
SO-906     Credit Card          Settled                 890.00
SO-907     UPI                  Settled                 460.00
SO-908     Debit Card           Pending                 380.00
SO-909     Net Banking          Failed                  240.00
SO-910     UPI                  Settled                 560.00

10 rows selected.
```

```
SQL> SELECT * FROM VW_DEPLETED_INVENTORY_ITEMS;

ITEM_CODE  ITEM_NAME                       AVAILABLE_STOCK
---------- ------------------------------ ---------------
ITM-107    Noise-Canceling Earbuds                      4
ITM-110    Artisanal Herbal Tea                         0
```

```
SQL> SELECT * FROM VW_DAILY_TURNOVER_SUMMARY;

SALES_DAT TOTAL_ORDERS TOTAL_REVENUE
--------- ------------ -------------
01-SEP-26            1        560.00
02-SEP-26            1        260.00
03-SEP-26            1        650.00
04-SEP-26            1        420.00
05-SEP-26            1        190.00
06-SEP-26            1        890.00
07-SEP-26            1        460.00
08-SEP-26            1        380.00
09-SEP-26            1        240.00
10-SEP-26            1        560.00

10 rows selected.
```

```
SQL> SELECT * FROM VW_MONTHLY_REVENUE_AUDIT;

SALES_M TOTAL_ORDERS TOTAL_REVENUE
------- ------------ -------------
2026-09           10       4610.00
```

```
SQL> SELECT * FROM VW_SHOPPER_LIFETIME_EXPENDITURE;

SHOPPER_ID SHOPPER_NAME                   TOTAL_ORDERS TOTAL_SPENDING
---------- ------------------------------ ------------ --------------
SHP-201    Aarav Sharma                              1         560.00
SHP-202    Meera Krishnan                            1         260.00
SHP-203    Rohan Desai                               1         650.00
SHP-204    Pooja Nair                                1         420.00
SHP-205    Vikram Sengupta                           1         190.00
SHP-206    Ananya Iyer                               1         890.00
SHP-207    Karthik Raman                             1         460.00
SHP-208    Divya Nambiar                             1         380.00
SHP-209    Siddharth Verma                           1         240.00
SHP-210    Tanvi Kulkarni                            1         560.00

10 rows selected.
```

Procedures Testing:

- Test procedure execution.
- Verify input parameters.
- Check output results.

```
SQL> EXEC SP_REGISTER_SALES_TRANSACTION('SO-911', 'SHP-201', 'ITM-101', 2, 'TXN-711', 'UPI');

PL/SQL procedure successfully completed.
```

```
SQL> EXEC SP_UPDATE_FULFILMENT_STATE('SO-911', 'Shipped');

PL/SQL procedure successfully completed.
```

```
SQL> VARIABLE rc REFCURSOR;
SQL> EXEC SP_RETRIEVE_SHOPPER_ORDERS('SHP-201', :rc);

PL/SQL procedure successfully completed.

SQL> PRINT rc;

SHOPPER_NAME                   ORDER_NUM  BOOKING_D ORDER_TOTAL FULFILMENT_
------------------------------ ---------- --------- ----------- -----------
Aarav Sharma                   SO-901     01-SEP-26      560.00 Delivered
Aarav Sharma                   SO-911     05-NOV-26      560.00 Shipped
```

```
SQL> EXEC SP_INWARD_INVENTORY_RESTOCK('ITM-101', 10);

PL/SQL procedure successfully completed.
```

Functions Testing:

- Verify calculation accuracy.
- Check returned values.

```
SQL> SELECT FN_COMPUTE_INTERVAL_REVENUE(DATE '2026-01-01', DATE '2026-12-31') AS TOTAL_SALES FROM DUAL;

TOTAL_SALES
-----------
       5170
```

```
SQL> SELECT FN_GET_SHOPPER_EXPENDITURE('SHP-201') AS TOTAL_SPENDING FROM DUAL;

TOTAL_SPENDING
--------------
          1120
```

```
SQL> SELECT FN_EVALUATE_ITEM_REVENUE('ITM-101') AS PRODUCT_REVENUE FROM DUAL;

PRODUCT_REVENUE
---------------
           1910
```